

Project Report

Nicholas Shvelidze + Chaitanya Anand

1. Description of the Project

KeyStonePA is a web-based machine learning application designed around the “Best in PA” theme. The purpose of the project is to help users identify potentially undervalued homes in Pennsylvania. Instead of simply showing houses by price, the application estimates whether a listing may be a good deal by comparing its price per square foot against market patterns learned from previously collected Pennsylvania housing data.

The user enters a Pennsylvania ZIP code and preferences such as maximum price, minimum number of bedrooms, minimum bathrooms, and minimum square footage. The backend then uses Playwright to scrape current Redfin listings for that ZIP code. Each listing is processed and scored using a price-per-square-foot valuation method. The application returns the top 10 homes that best match the user’s preferences and labels each listing as an Excellent Steal, Good Deal, Fair Deal, Fair Price, or Possibly Overpriced.

2. Significance of the Project

Housing search websites usually provide many listings but do not clearly explain whether a property is truly a good deal. A house may look affordable, but it may still be overpriced for its size or location. This project addresses that problem by using machine learning and feature engineering to compare homes more intelligently.

The novelty of this project is the “deal score” concept. The application does not only search for homes; it estimates what a home should cost based on local market price-per-square-foot patterns. This makes the tool useful for buyers who want a quick, explainable way to compare housing value across Pennsylvania ZIP codes.

3. Data Collection

Housing data was collected from Redfin using Playwright web scraping. The scraper collected listing information from multiple Pennsylvania ZIP codes. The collected fields included:

- Beds
- Baths

SQFT
Price
Zip_Code
Address
URL

The dataset was stored as a CSV file. The cleaned dataset contained approximately 406 valid housing records after removing incomplete or unrealistic values. The project originally tested a smaller dataset, but the model performance was weak because there were too few examples per ZIP code. The dataset was then expanded across more Pennsylvania ZIP codes to improve model quality.

Sample data format:

Beds,Baths,SQFT,Price,Zip_Code,Address,URL
3,2.5,1758,399900,18966,"Example Address, PA 18966","https://www.redfin.com/..."
4,2.5,2860,675000,19038,"Example Address, PA 19038","https://www.redfin.com/..."

4. Data Processing and Feature Engineering

The raw housing data was cleaned before training. Missing values were removed, and numeric columns were converted into proper numeric format. Unrealistic outliers were filtered out, including invalid bedroom counts, bathroom counts, square footage, and prices.

The most important engineered feature was:

$\text{Price_Per_SQFT} = \text{Price} / \text{SQFT}$

This feature is important because price alone does not explain value. For example, a \$500,000 house with 3,000 square feet may be a better deal than a \$400,000 house with 1,200 square feet. Price per square foot normalizes the comparison.

The project also created region labels from ZIP codes, such as:

Philadelphia / Southeast PA Area
Pittsburgh Area
Central PA
Lehigh Valley / Scranton Area
State College Area

This allowed the model to make broader regional predictions rather than relying only on exact ZIP code classification.

5. Model Development

The project used multiple machine learning components.

First, a Logistic Regression model was trained to classify the housing region based on:

- Beds
- Baths
- SQFT
- Price
- Price_Per_SQFT

The region classification model achieved approximately 46% accuracy. This result shows that house features alone can somewhat indicate the region, but many Pennsylvania housing markets overlap.

Second, a Random Forest Regression model was trained to predict housing price. This model performed much better, achieving:

- Mean Absolute Error: about \$27,464
- R² Score: about 0.975

This means the regression model captured pricing patterns strongly in the collected dataset.

Third, the project created a deal scoring method based on price per square foot:

- Estimated Market Value = Market Average Price_Per_SQFT x House SQFT
- Estimated Savings = Estimated Market Value – Listing Price
- Savings Percent = Estimated Savings / Estimated Market Value x 100

This became the core inference method used in the web application.

6. Showcase

The final application allows the user to search for houses in any Pennsylvania ZIP code. The user enters:

- ZIP code
- Maximum price
- Minimum beds
- Minimum baths
- Minimum square footage

The backend scrapes matching Redfin listings, calculates price per square foot, estimates market value, and returns the top 10 homes ranked by deal score.

Each result includes:

- Address
- Price
- Beds
- Baths
- Square feet
- Price per square foot
- Estimated savings
- Savings percentage
- Deal label
- Listing link
- Sales pitch explanation

The frontend also supports saving listings and comparing saved homes.

7. Discussion and Conclusion

The project successfully demonstrates the use of machine learning, web scraping, feature engineering, and web application development in a real-world housing search problem. The strongest part of the system is the price prediction and deal scoring logic. The price regression model performed well, while the region classification model had moderate performance.

One challenge was web scraping. Real estate websites use JavaScript and bot detection, so Playwright had to be used with delays and visible browsing. Another issue was that scraped text sometimes included non-listing prices such as fees, which required guardrails to remove unrealistic values.

A limitation of the current system is that it does not account for all real estate factors. Home condition, school district, lot size, taxes, year built, renovations, and neighborhood quality can significantly affect price. However, for a class project, the system provides a strong and explainable starting point for housing deal analysis.

8. AI Usage

AI tools were used to assist with project planning, code debugging, documentation drafting, and explanation of machine learning concepts. AI helped generate starter code for the FastAPI backend, React frontend structure, Playwright scraping logic, and project report wording.

AI did not replace the project work entirely. The final system required manual testing, debugging, code integration, dataset collection, model training, and adjustment of project logic. AI was used as a development assistant, while the final project decisions and implementation were controlled by the project team.